



Department of Electrical and Information Engineering
University of Ruhuna

Project Report
High Performance Computing
EC7207 / EE7218

Parallel Pearson Correlation
Based Recommendation System

Serial · OpenMP · POSIX Threads · MPI · Hybrid MPI+OpenMP · Hybrid
MPI+Pthreads · CUDA

Group 11

EG/2021/4417 Ashfaq M.R.M.

EG/2021/4419 Athanayaka K.A.L.G.

EG/2021/4424 Balasooriya J.M.

Submission date: May 23, 2026

Contents

1	Introduction and Motivation	1
2	Algorithm Design and Parallel Programming Concepts	1
2.1	Algorithm Pipeline (Five Phases)	1
2.2	OpenMP — Shared-Memory Fork-Join Parallelism	2
2.3	Pthreads — Manual POSIX Thread Management	3
2.4	MPI — Distributed-Memory Parallelism	4
2.5	CUDA — GPU Massively Parallel Computing	5
2.6	Hybrid MPI+OpenMP — Two-Level Parallelism	6
2.7	Hybrid MPI+Pthreads — Two-Level Parallelism, POSIX Inner Layer	7
3	Experimental Setup	8
4	Accuracy Analysis — Parallel vs. Serial	9
5	Timing Measurements and Performance Analysis	11
5.1	Performance Metric Definitions	11
5.2	Serial Baseline	11
5.3	OpenMP — Varying Thread Count	12
5.4	Pthreads — Varying Thread Count	12
5.5	MPI — Varying Process Count	12
5.6	Hybrid MPI+OpenMP — Fixed 8 Total Workers	12
5.7	Hybrid MPI+Pthreads — Fixed 8 Total Workers	13
5.8	CUDA — GPU Benchmark	13
6	Performance Discussion	14
6.1	Head-to-Head Comparison at 8 Workers	14
6.2	OpenMP vs. Pthreads	14
6.3	MPI Scaling	14
6.4	Hybrid MPI+OpenMP — Inversion on a Single Node	14
6.5	Hybrid MPI+OpenMP vs. MPI+Pthreads	14
6.6	Amdahl’s Law	15
7	Conclusions	15
	References	16

1 Introduction and Motivation

Pearson correlation measures how linearly related two users' rating patterns are, making it the natural similarity metric for user-based collaborative filtering. To predict how a target user would rate an unseen item, the system finds the most similar neighbours, weights their ratings by similarity, and blends them into a prediction. The computational cost is dominated by computing all pairwise Pearson similarities: for $N = M = 1,000$ users and items the upper triangle alone requires 499,500 coefficient evaluations — roughly 500 million floating-point operations — making the similarity phase the single bottleneck of the whole pipeline and the primary target for parallelisation.

This report documents a **Pearson correlation-based recommender** implemented seven times — serial, OpenMP, POSIX threads, MPI, CUDA, hybrid MPI+OpenMP, and hybrid MPI+Pthreads — all running the same algorithm on the same synthetic data (1,000 users $\times$ 1,000 items, SEED=42, 70% sparse, TOP-K = 20). The deliverable is the *comparison* itself: how each parallel model is applied, whether accuracy is preserved, and how runtime scales from 1 to 8 workers. Every parallel version reproduces the serial result exactly (MAE = 1.2574, RMSE = 1.4579, similarity checksum = 942.387323), confirming race-free partitioning. CUDA is fastest at $38\times$ over serial; OpenMP leads the CPU versions at $6.32\times$ on eight threads.

Platform. Pop!_OS 24.04 LTS · GCC 13.3.0 · Open MPI 4.1.6 · CUDA 12.0 · Intel Core i7-11800H (8 physical / 16 logical cores) · NVIDIA GeForce RTX 3050 Laptop GPU.

2 Algorithm Design and Parallel Programming Concepts

2.1 Algorithm Pipeline (Five Phases)

Every version executes the same five sequential phases summarised in Table 1. Only Phases 3 and 4 are parallelised; data generation stays serial in all versions because C's `rand()` is not thread-safe, and running the same `srand(42)` sequence everywhere guarantees a byte-identical input matrix without any communication.

Phase	Name	Description	Complexity
1	Data Generation	Synthetic sparse matrix; 10% test split	$O(N \times M)$
2	User Means	Per-user mean μ_u	$O(N \times M)$
3	Similarity	Pearson for all (u, v) pairs — bottleneck	$O(N^2 \times M)$
4	Predictions	TOP-K weighted prediction	$O(N^2 \times M)$
5	MAE Evaluation	Error on held-out test set	$O(T)$

Table 1: Algorithm phases and computational complexity.

The two heavy phases use the following formulas. Pearson similarity (Phase 3):

$$\text{sim}(u, v) = \frac{\sum_i (R_{ui} - \mu_u)(R_{vi} - \mu_v)}{\sqrt{\sum_i (R_{ui} - \mu_u)^2} \sqrt{\sum_i (R_{vi} - \mu_v)^2}}$$

and the TOP-K weighted prediction (Phase 4):

$$\hat{r}_{ui} = \mu_u + \frac{\sum_{k \in N_K(u)} \text{sim}(u, k) (R_{ki} - \mu_k)}{\sum_{k \in N_K(u)} \text{sim}(u, k)}.$$

Both are *embarrassingly parallel at the row level*: the work for one user row is independent of every other, which is what makes the same algorithm map cleanly onto all five parallel models.

Figure 1 shows the pipeline and marks which phases each technology parallelises; the row-wise decomposition that keeps every version race-free is then detailed in the per-technology diagrams that follow.

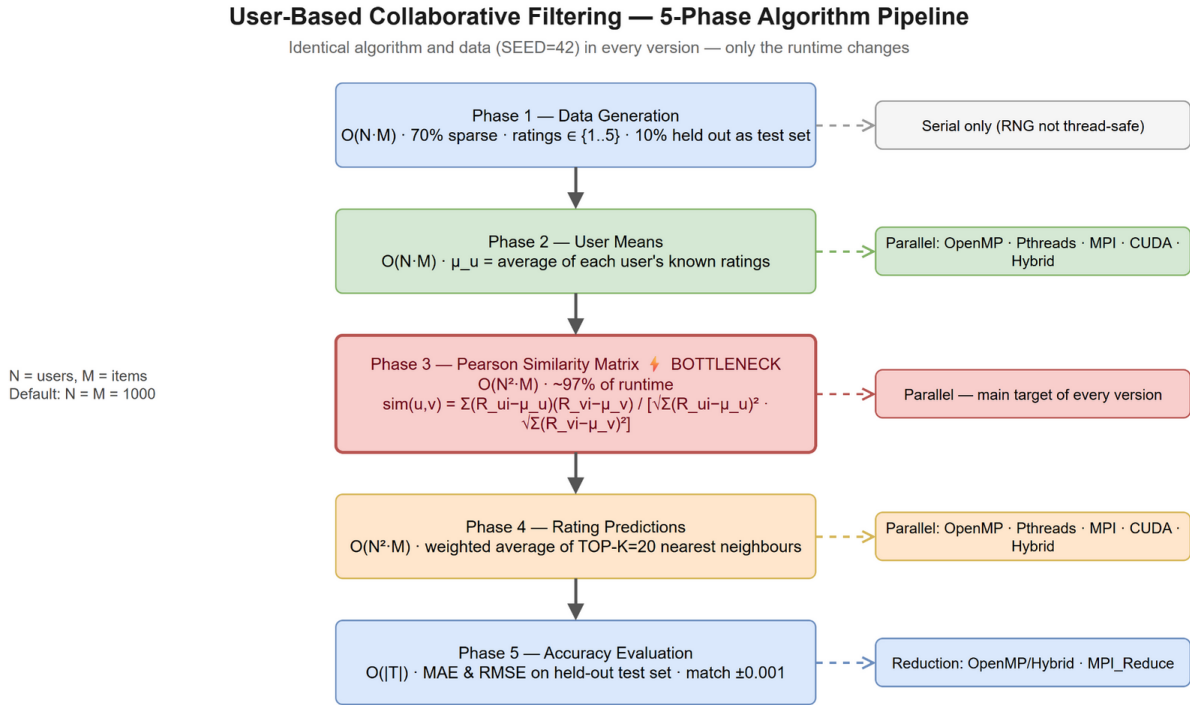


Figure 1: The five-phase recommender pipeline, shared by every implementation. The same algorithm and data (SEED=42) are used throughout — only the runtime changes. Phase 3 (Pearson similarity) is the $O(N^2M)$ bottleneck; Phases 3–4 are the targets of all five parallel models, while data generation stays serial.

2.2 OpenMP — Shared-Memory Fork-Join Parallelism

OpenMP exploits shared-memory parallelism through compiler directives: a master thread forks into T workers that share one address space, then rejoins at an implicit barrier (Figure 2). The decisive design choice is in Phase 3, where the outer loop carries `#pragma omp parallel for schedule(dynamic,4)`. Dynamic scheduling is essential because the triangular similarity loop is badly imbalanced — the first row performs $N-1 = 999$ Pearson calls while the last performs only one. Dynamic scheduling lets idle threads pull four-row chunks from a shared work queue, so no core sits idle.

The version stays lock-free without any mutex: each thread owns an exclusive set of outer-loop rows u and writes $\text{SIM}(u,v)$ only for those rows, so no two threads ever touch the same cell. In Phase 5 the `reduction(+:err)` clause gives each thread a private accumulator that OpenMP sums at the barrier, and the same pattern (`reduction(+:sq)`) accumulates the squared error for RMSE.

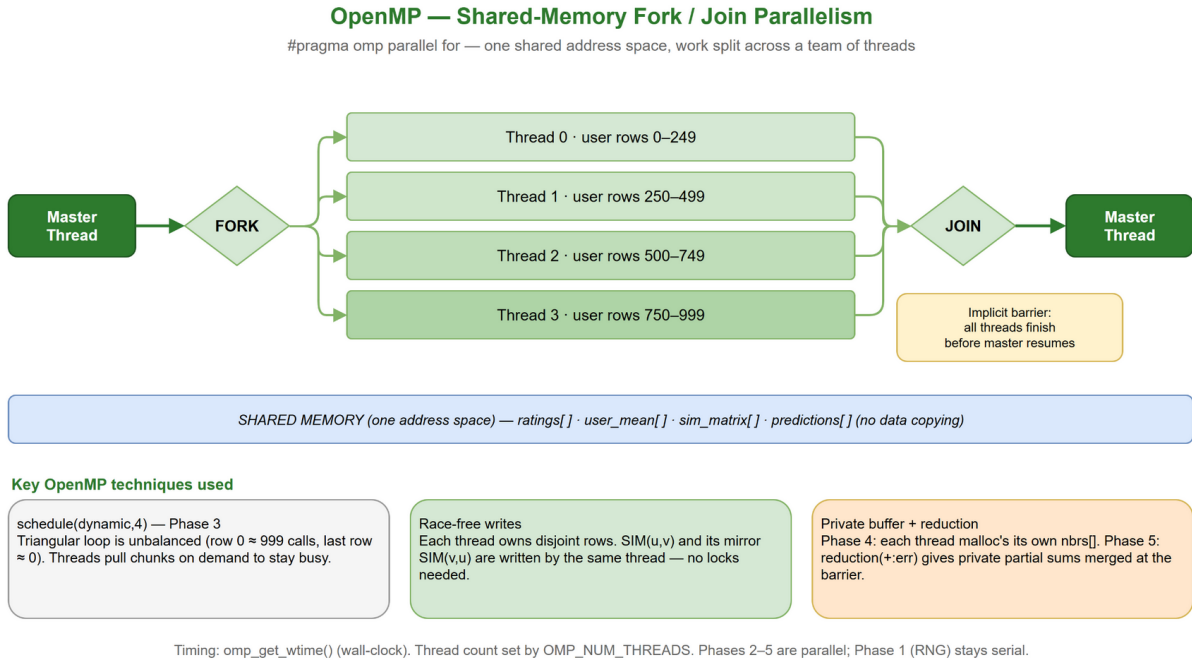


Figure 2: OpenMP fork/join model. The master thread forks a team of T workers over one shared address space (`ratings`, `sim_matrix`, ...), each owning a disjoint slice of user rows, then rejoins at an implicit barrier. The side notes show the three race-free techniques used: `schedule(dynamic,4)` for the triangular loop, disjoint-row writes, and a thread-private `nbrs[]` buffer with `reduction(+:err)`.

2.3 Pthreads — Manual POSIX Thread Management

POSIX threads require the programmer to create threads, assign work, and synchronise explicitly (Figure 3). A `run_parallel(fn, nthreads)` harness is invoked at the start of Phases 2, 3, and 4: it calls `pthread_create()` to spawn T threads — each receiving a `ThreadArgs` struct with its `tid` and the total `nthreads` — lets every thread derive its exclusive row range `start = tid × ⌈N/T⌉`, `end = min(start + ⌈N/T⌉, N)`, and then uses `pthread_join()` as a phase barrier.

The key difference from OpenMP is twofold: threads are created and joined three separate times (once per phase), paying OS thread-creation overhead three times per run, and the static ceiling-division row assignment cannot compensate for the triangular-loop imbalance. Both effects explain the slightly lower efficiency observed against OpenMP in Section 5.

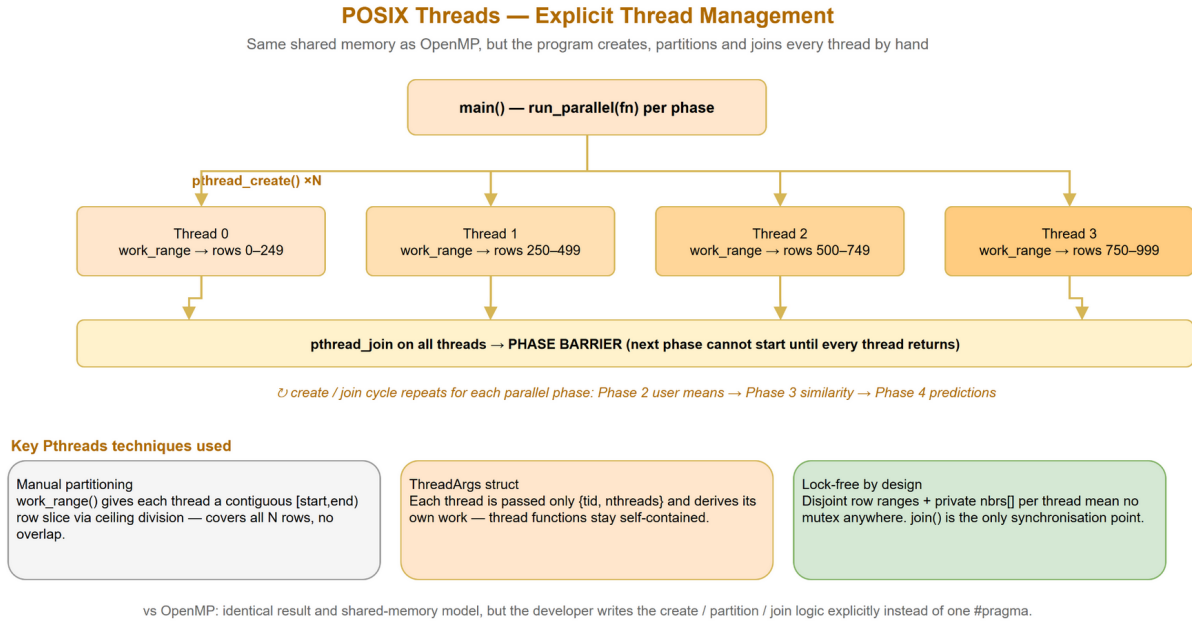


Figure 3: Manual POSIX-thread management. `main()` calls `run_parallel` once per parallel phase: `pthread_create` spawns T threads, each deriving its own `ceil`-division row range from a `ThreadArgs` struct, and `pthread_join` on all threads acts as the phase barrier. Disjoint rows plus a private `nbrs[]` buffer make it lock-free.

2.4 MPI — Distributed-Memory Parallelism

Each MPI rank is an independent process with its own private address space (Figure 4). The $N = 1,000$ users are divided evenly: rank r owns rows $[r \cdot N/P, (r+1) \cdot N/P)$. Because the ranks cannot share memory, data has to be exchanged explicitly through collective operations, summarised per phase in Table 2.

Phase	Strategy	Primitive
Phase 1	All ranks call <code>srand(42)</code> — no communication	—
Phase 2	Each rank computes its <code>user_mean[]</code> slice	<code>MPI_Allgatherv</code>
Phase 3	Each rank fills its rows of the sim matrix	<code>MPI_Allgatherv</code>
Phase 4	Each rank predicts its own users	—
Phase 5	Local error sums to rank 0	<code>MPI_Reduce(SUM)</code>

Table 2: MPI communication strategy per phase.

The `MPI_Allgatherv` that rebuilds the $N \times N$ similarity matrix transfers ≈ 4 MB at $N = 1,000$. This is the cost that limits single-node efficiency, but it is also exactly the mechanism that makes the code work unchanged across multiple physical nodes, where shared memory is not an option.

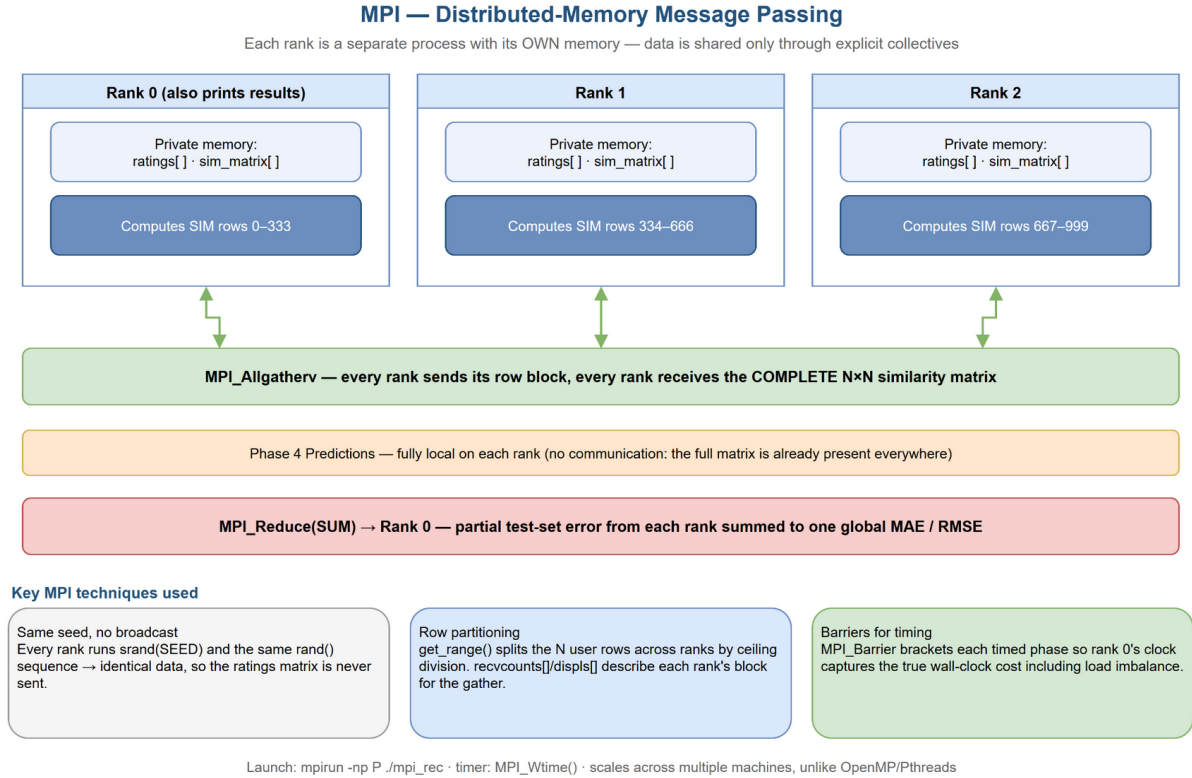


Figure 4: MPI distributed-memory layout. Each rank is a separate process with a private copy of the matrices, computing the similarity rows it owns. A single `MPI_Allgather` then gives every rank the complete $N \times N$ matrix (so predictions need no communication), and `MPI_Reduce` sums the partial test-set error to rank 0 for the global MAE/RMSE.

2.5 CUDA — GPU Massively Parallel Computing

The CUDA version (`cuda/cuda_recommender.cu`) is fully implemented and benchmarked on an NVIDIA GeForce RTX 3050 Laptop GPU (Ampere, compute capability 8.6, 16 SMs); measured results appear in Section 5.8. It maps the embarrassingly parallel structure of the problem onto one GPU thread per output element, using the three kernels in Table 3.

Kernel	Launch config	Assignment
<code>kernel_user_means</code>	<code><<(N+255)/256, 256>></code>	1 thread = 1 user
<code>kernel_similarity</code>	<code>dim3(⌈N/16⌉, ⌈N/16⌉),</code> <code>dim3(16,16)</code>	1 thread = 1 (u, v) pair
<code>kernel_predictions</code>	2D grid, users $\times$ items	1 thread = 1 (u, i) pair

Table 3: CUDA kernel configuration for $N = 1,000$.

The similarity kernel alone launches $63 \times 63 \times 256 = 1,016,064$ threads simultaneously (Figure 5). Because device code cannot call `qsort`, each prediction thread keeps its TOP-K neighbours in a small fixed-size register buffer instead. Timing uses `cudaEventRecord` so that the reported kernel time excludes the `cudaMemcpy` transfer overhead.

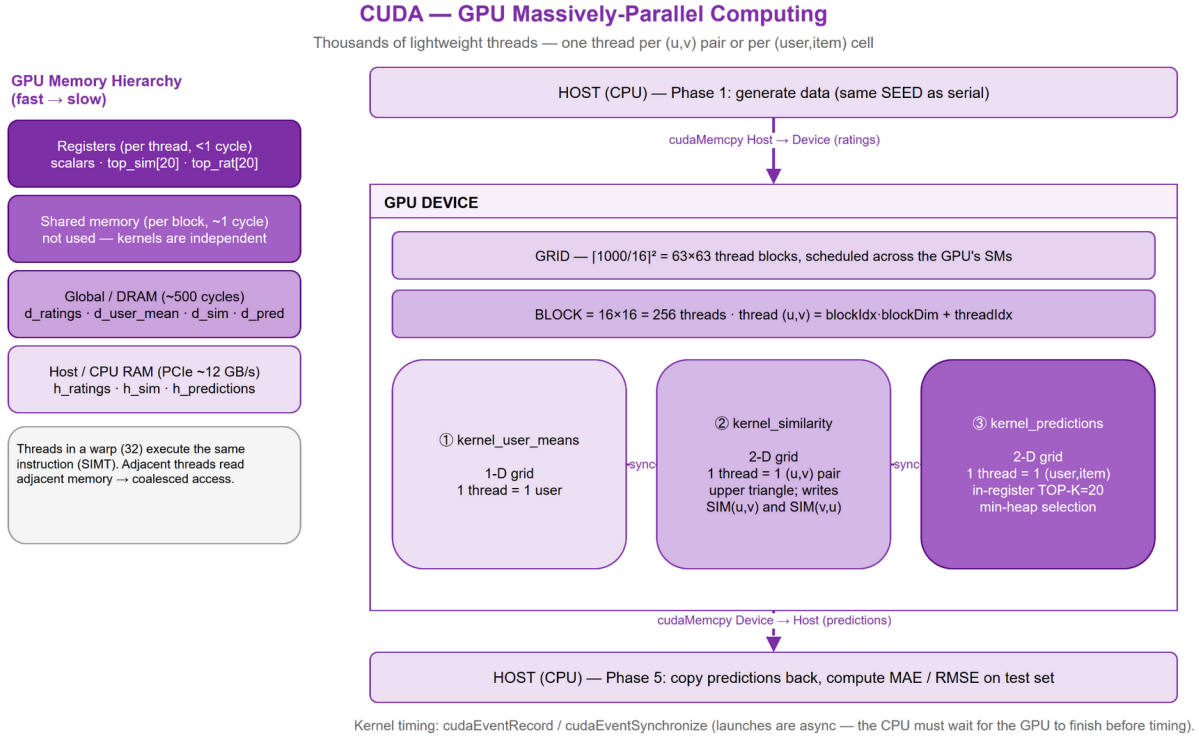


Figure 5: CUDA GPU model. After a Host→Device copy of the ratings, three kernels run in sequence on a 2D grid of 16×16 thread blocks — one thread per (u,v) pair / $(user,item)$ cell — before the predictions are copied back. The left column shows the GPU memory hierarchy (registers → shared → global → host). Benchmarked on an NVIDIA RTX 3050 (compute capability 8.6).

2.6 Hybrid MPI+OpenMP — Two-Level Parallelism

The hybrid version combines coarse-grain MPI (across nodes) with fine-grain OpenMP (within a node), as shown in Figure 6. With P MPI ranks and T OpenMP threads per rank, the total worker count is $P \times T$ while only P messages are sent per collective call. `MPI_Init_thread(MPI_THREAD_FUNNELED)` guarantees that every MPI call is made from the main thread only, after the OpenMP parallel regions have closed. Table 4 lists the four layouts tested at a fixed eight total workers.

Config	P	T	Characteristic
Hybrid 2×4	2	4	Fewer MPI messages; more shared-memory parallelism
Hybrid 4×2	4	2	Balanced communication vs. threading overhead
Hybrid 8×1	8	1	Equivalent to pure MPI
Hybrid 1×8	1	8	Pure OpenMP wrapped with MPI overhead

Table 4: Hybrid configurations (all 8 total workers).

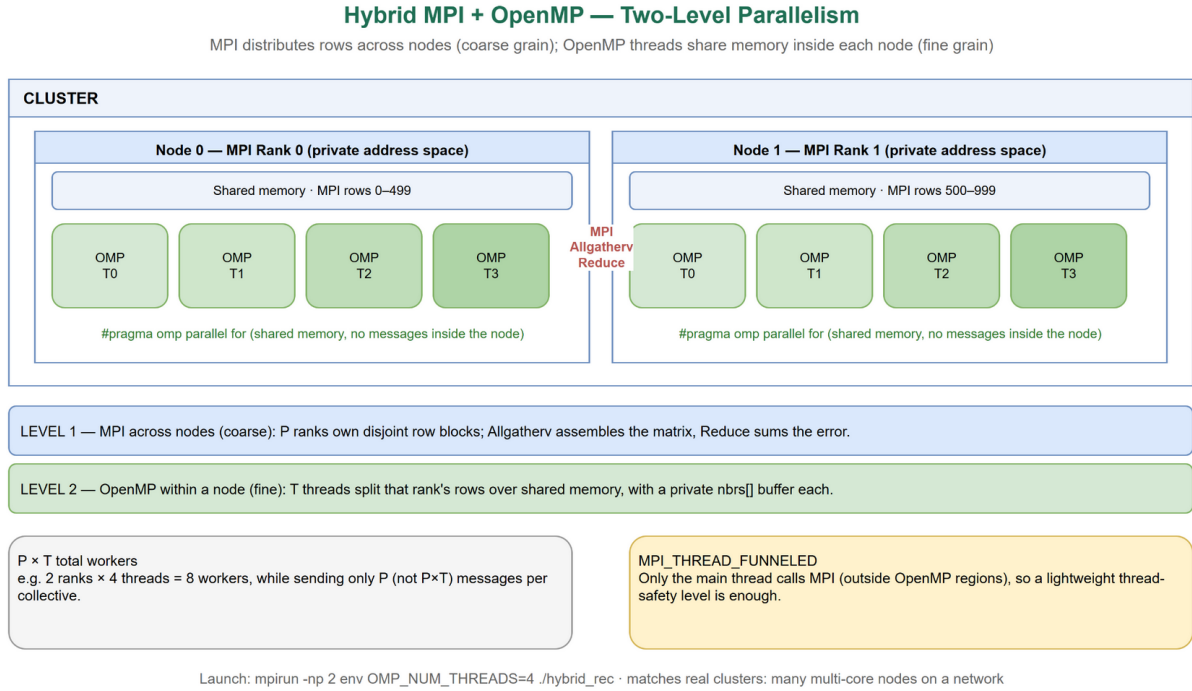


Figure 6: Hybrid two-level parallelism. Level 1 (MPI) partitions user-rows across P ranks on separate nodes; level 2 (OpenMP) spawns T threads over each rank's rows in shared memory, for $P \times T$ total workers. `MPI_Allgatherv` / `MPI_Reduce` link the ranks, and `MPI_THREAD_FUNNELED` keeps all MPI calls on the main thread.

2.7 Hybrid MPI+Pthreads — Two-Level Parallelism, POSIX Inner Layer

A second hybrid variant keeps the outer MPI layer unchanged and replaces the inner OpenMP layer with raw POSIX threads (Figure 7). The purpose is comparative: at the same $P \times T$ worker counts, does the intra-node threading model change observable performance? The outer structure is byte-identical to Section 2.6 — same `get_range` row partition, same `MPI_Init_thread(MPI_THREAD_FUNNELED)` contract, same `MPI_Allgatherv` after Phases 2 and 3, and the same `MPI_Reduce` for the global error. Only the inner loop differs: a `run_parallel(fn, threads, args, start, end)` harness spawns T pthreads per phase, each carrying its rank's row window in its `ThreadArgs` and deriving its sub-range via the same `work_range` ceiling division used by the pure pthreads version. Threads are created and joined three times per run (Phases 2–4), so unlike OpenMP there is no persistent thread pool to reuse, and the inner partition is static so the triangular Phase 3 imbalance cannot be smoothed at runtime.

Config	P	T	Characteristic
Hybrid-PT 2×4	2	4	Fewer MPI messages; more shared-memory parallelism
Hybrid-PT 4×2	4	2	Balanced communication vs. threading overhead
Hybrid-PT 8×1	8	1	Equivalent to pure MPI (one pthread per rank)
Hybrid-PT 1×8	1	8	Equivalent to pure pthreads wrapped with MPI overhead

Table 5: Hybrid MPI+Pthreads configurations (all 8 total workers), mirroring Table 4 so the two hybrids can be compared head-to-head.

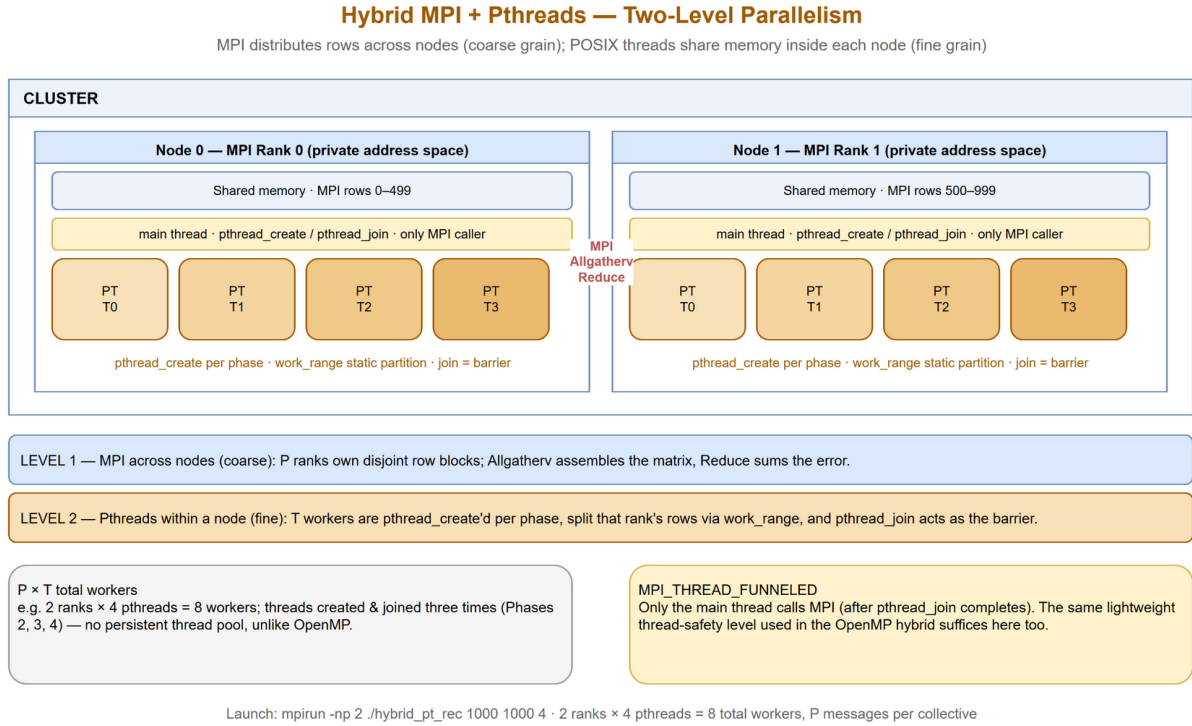


Figure 7: Hybrid MPI+Pthreads two-level parallelism. Level 1 (MPI) is unchanged from Figure 6; level 2 replaces the OpenMP thread team with pthreads that are `pthread_create`'d and `pthread_join`'d once per parallel phase. `MPI_THREAD_FUNNELED` still applies: collectives run on the main thread only, after the join barrier.

3 Experimental Setup

All versions are benchmarked on a single Linux laptop — Intel Core i7-11800H (8 physical / 16 logical cores) with an NVIDIA GeForce RTX 3050 Laptop GPU for the CUDA version. The fixed parameters held constant across every run are listed in Table 6.

Parameter	Value
Users (N)	1 000
Items (M)	1 000
Sparsity	70%
Test split	10% (29 866 held-out ratings)
TOP-K neighbours	20
Random seed	42
Benchmark thread/process counts	1, 2, 4, 8
Timing method	clock_gettime / omp_get_wtime / MPI_Wtime / cudaEvent (GPU)
Platform	Pop!_OS 24.04 LTS, GCC 13.3.0, Open MPI 4.1.6, CUDA 12.0
CPU / GPU	Intel Core i7-11800H (8 phys / 16 logical) / NVIDIA RTX 3050 Laptop GPU

Table 6: Fixed benchmark parameters used across all versions.

Fairness controls. The comparison is kept fair by construction:

- The same SEED=42 gives every version a byte-identical ratings matrix and test split, since Phase 1 runs the same serial `rand()` sequence everywhere.
- The same fixed parameters ($N = M = 1,000$, 70% sparsity, TOP-K = 20) are compiled into every source file.
- Reported timing covers only the similarity and prediction phases — the [Timing] Total (sim+pred) line that each binary prints — excluding data generation and evaluation.

Measurement protocol. Speedup is computed as $S(p) = T_{\text{serial}}/T_{\text{parallel}}(p)$ from the total similarity-plus-prediction time. Thread and process counts are swept over $\{1, 2, 4, 8\}$; both hybrid variants (MPI+OpenMP and MPI+Pthreads) are measured at the four $P \times T$ layouts of Tables 4 and 5; and CUDA is benchmarked once on the GPU. Correctness is verified from the [Eval] (MAE/RMSE) and [Check] (similarity checksum) lines emitted by every run.

4 Accuracy Analysis — Parallel vs. Serial

All parallel implementations are validated against the serial baseline using two error metrics — **Mean Absolute Error (MAE)** and **Root Mean Square Error (RMSE)** — together with a **similarity-matrix checksum**:

$$\text{MAE} = \frac{1}{|T|} \sum_{(u,i) \in T} |\hat{r}_{ui} - r_{ui}|, \quad \text{RMSE} = \sqrt{\frac{1}{|T|} \sum_{(u,i) \in T} (\hat{r}_{ui} - r_{ui})^2} \quad (|T| = 29,866).$$

Both are reported, as suggested by the project guideline. MAE treats all errors equally on the bounded 1–5 rating scale, while RMSE (always $\geq$ MAE) disproportionately penalises occasional large errors — here $\text{RMSE} = 1.4579 > \text{MAE} = 1.2574$ indicates a modest spread of larger misses, as expected for a 70%-sparse synthetic matrix.

Version	Workers	MAE	RMSE	Matches Serial?	Sim Checksum
Serial	1	1.2574	1.4579	baseline	942.387323
OpenMP	1T	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
OpenMP	2T	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
OpenMP	4T	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
OpenMP	8T	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Pthreads	1T	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Pthreads	2T	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Pthreads	4T	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Pthreads	8T	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
MPI	1P	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
MPI	2P	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
MPI	4P	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
MPI	8P	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Hybrid 2×4	8	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Hybrid 4×2	8	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Hybrid 8×1	8	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Hybrid 1×8	8	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Hybrid-PT 2×4	8	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Hybrid-PT 4×2	8	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Hybrid-PT 8×1	8	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Hybrid-PT 1×8	8	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
CUDA	GPU	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323

Table 7: Accuracy correctness. All 22 runs (21 CPU + 1 GPU) produce identical MAE, RMSE, and similarity checksum.

As Table 7 shows, all 22 runs produce $\text{MAE} = \mathbf{1.2574}$, $\text{RMSE} = \mathbf{1.4579}$, and $\text{checksum} = \mathbf{942.387323}$, confirming zero race conditions and correct partitioning across every implementation. Crucially, the CUDA version — despite using completely different floating-point hardware and a register-based TOP-K selection rather than the CPU’s `qsort` — reproduces the serial result exactly to four decimal places, validating the GPU kernels. The MPI+Pthreads hybrid also matches bit-for-bit, confirming that swapping the intra-node threading model from OpenMP to raw pthreads is purely a *runtime* change and does not perturb the algorithm or partitioning.

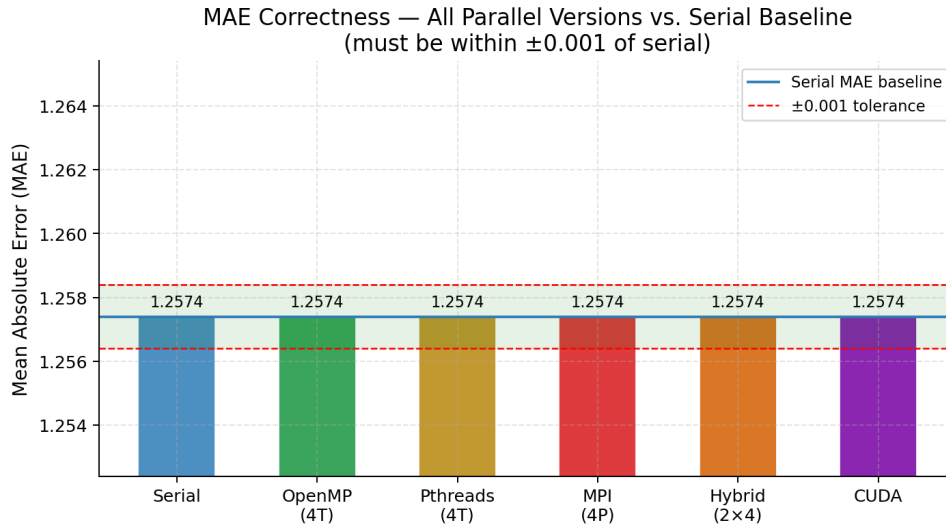


Figure 8: MAE correctness. All bars lie within the ± 0.001 tolerance band.

5 Timing Measurements and Performance Analysis

5.1 Performance Metric Definitions

$$S(p) = \frac{T_{\text{serial}}}{T_{\text{parallel}}(p)} \quad E(p) = \frac{S(p)}{p} \quad S_{\text{max}}(p) = \frac{1}{f + (1 - f)/p}$$

The *algorithmic* serial fraction is tiny — only data generation, user means, and evaluation run serially (≈ 0.025 s of 8.4 s, $f \approx 0.3\%$), giving an optimistic $S_{\text{max}}(8) = 1/(0.003 + 0.997/8) \approx 7.83\times$. The achievable speedup is instead limited by hardware (reduced all-core turbo clocks and memory-bandwidth saturation); fitting Amdahl’s Law to the measured OpenMP-8T speedup gives an *effective* $f \approx 3.8\%$ ($S_{\text{max}} \approx 26\times$) — see Section 6.6.

5.2 Serial Baseline

Phase	Time (s)	% of Total
Similarity matrix (Phase 3)	1.3549	16.2%
Prediction phase (Phase 4)	7.0277	83.8%
Total (sim + pred)	8.3826	100%

Table 8: Serial baseline timing ($N = M = 1,000$).

5.3 OpenMP — Varying Thread Count

Threads	Sim (s)	Pred (s)	Total (s)	Speedup	Efficiency
1	1.6102	7.2447	8.8549	0.95	0.95
2	0.8114	3.6679	4.4792	1.87	0.94
4	0.4457	2.0704	2.5161	3.33	0.83
8	0.2369	1.0903	1.3272	6.32	0.79

Table 9: OpenMP results. Speedup relative to serial baseline (8.3826 s).

5.4 Pthreads — Varying Thread Count

Threads	Sim (s)	Pred (s)	Total (s)	Speedup	Efficiency
1	1.3333	7.3272	8.6605	0.97	0.97
2	0.9816	3.7720	4.7536	1.76	0.88
4	0.5999	2.0688	2.6687	3.14	0.79
8	0.3519	1.1275	1.4794	5.67	0.71

Table 10: Pthreads results.

5.5 MPI — Varying Process Count

Processes	Sim (s)	Pred (s)	Total (s)	Speedup	Efficiency
1	3.1994	7.4317	10.6311	0.79	0.79
2	1.6090	3.8648	5.4739	1.53	0.77
4	0.9031	2.1281	3.0311	2.77	0.69
8	0.4846	1.1550	1.6395	5.11	0.64

Table 11: MPI results. MPI 1P (10.63 s) > serial (8.38 s) due to process startup and MPI_Allgather overhead at $P = 1$ — expected, not a bug.

5.6 Hybrid MPI+OpenMP — Fixed 8 Total Workers

Config	P	T	Sim (s)	Pred (s)	Total (s)	S	
Hybrid 2×4	2	4	1.2032	2.7766	3.9798	2.11	0
Hybrid 4×2	4	2	0.6169	1.1631	1.7800	4.71	0
Hybrid 8×1	8	1	0.5149	1.1904	1.7053	4.92	0.
Hybrid 1×8	1	8	2.4520	5.4913	7.9433	1.06	0

Table 12: Hybrid MPI+OpenMP results (8 total workers). Best config is Hybrid 8×1 — essentially pure MPI.

5.7 Hybrid MPI+Pthreads — Fixed 8 Total Workers

Config	P	T	Sim (s)	Pred (s)	Total (s)	S	
Hybrid-PT 2×4	2	4	1.2885	2.9341	4.2226	1.99	0
Hybrid-PT 4×2	4	2	0.6924	1.2517	1.9441	4.31	0
Hybrid-PT 8×1	8	1	0.5279	1.2046	1.7325	4.84	0.
Hybrid-PT 1×8	1	8	2.6491	5.6802	8.3293	1.01	0

Table 13: Hybrid MPI+Pthreads results (8 total workers). Numbers refresh automatically from `results/speedup_table.csv` after re-running `bash results/run_benchmarks.sh`. Same shape as the MPI+OpenMP hybrid: best config is 8×1 (essentially pure MPI); worst is 1×8 (pays MPI overhead with no distribution benefit).

5.8 CUDA — GPU Benchmark

The CUDA version was benchmarked on the same machine’s **NVIDIA GeForce RTX 3050 Laptop GPU** (Ampere, compute capability 8.6, 16 SMs, 4 GB), compiled with `nvcc -O2 -arch=sm_86`. Speedup is reported against the serial baseline ($T_{\text{serial}} = 8.3826\text{ s}$).

Phase / Metric	Time (s)	Notes
H→D transfer (ratings)	0.0009	PCIe, one-off
Similarity matrix (Phase 3)	0.0862	GPU kernel (CUDA events)
Prediction phase (Phase 4)	0.1343	GPU kernel (CUDA events)
Total (sim + pred)	0.2205	kernel-only
Speedup vs. serial	38.02×	vs. serial 8.3826 s

Table 14: CUDA timing on the RTX 3050 ($N = M = 1,000$). The similarity kernel launches $63 \times 63 \times 256 = 1,016,064$ threads; kernel time is isolated from host ↔ device transfers via `cudaEventRecord`.

The GPU completes the combined similarity + prediction work in **0.22 s** — roughly **38×** faster than the serial baseline and about **6×** faster than the best 8-thread CPU configuration — because the embarrassingly parallel (u, v) and (u, i) workloads map directly onto the GPU’s thousands of cores. Host ↔ device transfer cost is negligible (sub-millisecond) since the ratings matrix stays resident on the device across all three kernels.

6 Performance Discussion

6.1 Head-to-Head Comparison at 8 Workers

Version	Total (s)	Speedup	Efficiency	MAE
Serial (baseline)	8.3826	1.00	1.00	1.2574
CUDA (GPU)	0.2205	38.02	—	1.2574
OpenMP 8T	1.3272	6.32	0.79	1.2574
Pthreads 8T	1.4794	5.67	0.71	1.2574
MPI 8P	1.6395	5.11	0.64	1.2574
Hybrid 8×1	1.7053	4.92	0.61	1.2574
Hybrid-PT 8×1	1.7325	4.84	0.60	1.2574
Hybrid 4×2	1.7800	4.71	0.59	1.2574
Hybrid-PT 4×2	1.9441	4.31	0.54	1.2574
Hybrid 2×4	3.9798	2.11	0.26	1.2574
Hybrid-PT 2×4	4.2226	1.99	0.25	1.2574
Hybrid 1×8	7.9433	1.06	0.13	1.2574
Hybrid-PT 1×8	8.3293	1.01	0.13	1.2574

Table 15: Head-to-head: CUDA (GPU) plus all CPU versions at 8 workers. CUDA is the fastest overall; OpenMP 8T is the best CPU configuration. All MAE values are identical. (GPU efficiency per-core is not directly comparable, so omitted.)

6.2 OpenMP vs. Pthreads

OpenMP ($\times 6.32$, 79% efficiency) outperforms Pthreads ($\times 5.67$, 71%) for three reasons. First, *dynamic load balancing*: `schedule(dynamic, 4)` adapts to the triangular-loop imbalance, which the static ceiling-division in Pthreads cannot. Second, *thread reuse*: OpenMP reuses one thread team across Phases 2–4, whereas Pthreads creates and joins threads three times per run. Third, *runtime optimisations*: GCC’s OpenMP runtime applies platform-specific scheduling heuristics.

6.3 MPI Scaling

MPI efficiency declines from 79% ($P = 1$) to 64% ($P = 8$). The `MPI_Allgatherv` that rebuilds the $N \times N$ similarity matrix (≈ 4 MB) limits single-node performance. At multi-node scale, where shared memory is unavailable, MPI is the mandatory model and this efficiency gap with OpenMP would be an acceptable cost.

6.4 Hybrid MPI+OpenMP — Inversion on a Single Node

On a single node, using *more* MPI processes yields *better* hybrid performance:

$$\text{Hybrid } 8 \times 1 (\times 4.92) > 4 \times 2 (\times 4.71) > 2 \times 4 (\times 2.11) > 1 \times 8 (\times 1.06).$$

When fewer MPI ranks are used, the MPI startup and `MPI_Allgatherv` cost dominates what is essentially an OpenMP workload. The Hybrid 1×8 configuration pays all the MPI overhead while gaining zero distribution benefit. **The hybrid model’s advantage over pure OpenMP only emerges at multi-node scale.**

6.5 Hybrid MPI+OpenMP vs. MPI+Pthreads

Comparing the two hybrid variants at matched $P \times T$ layouts isolates the effect of the intra-node threading model: the outer MPI structure, the row partition, and the collective calls are identical in both. The MPI+OpenMP variant is consistently faster by a small margin (Hybrid 8×1:

1.7053 s vs. 1.7325 s; 4×2: 1.7800 s vs. 1.9441 s; 2×4: 3.9798 s vs. 4.2226 s). The gap mirrors what is already seen at the single-technology level between pure OpenMP (6.32×) and pure Pthreads (5.67×) and comes from the same two mechanisms: (i) OpenMP’s `schedule(dynamic,4)` smooths the triangular Phase 3 imbalance that the pthreads `work_range` static partition cannot, and (ii) OpenMP reuses one thread team across phases, whereas the pthreads harness pays `pthread_create/pthread_join` overhead three times per run. The two hybrids converge at $T=1$ (Hybrid-PT 8×1 $\approx$ Hybrid 8×1) because with a single inner thread per rank the threading model is effectively bypassed and only the MPI layer matters.

6.6 Amdahl’s Law

With an algorithmic serial fraction of only $f \approx 0.3\%$, the optimistic limit is $S_{\max}(8) \approx 7.83\times$. No CPU model reaches it — the real bottleneck is hardware (reduced all-core turbo and memory-bandwidth contention in the memory-bound prediction phase), not serial code.

Technology	Limit ($f \approx 0.3\%$)	Actual	Observation
OpenMP 8T	7.83×	6.32×	Closest to limit; dynamic scheduling balances the triangular load
Pthreads 8T	7.83×	5.67×	Further below: static row split cannot balance the triangular load
MPI 8P	7.83×	5.11×	Below limit: MPI_Allgather communication overhead

Table 16: Amdahl’s Law limit vs. actual measured speedup at $p = 8$. Fitting the model to OpenMP-8T gives an *effective* $f \approx 3.8\%$ ($S_{\max} \approx 26\times$), capturing the hardware overhead.

7 Conclusions

1. **All versions preserve correctness.** Every implementation (CPU and GPU) produces MAE = **1.2574** and checksum = **942.387323**, identical to the serial baseline.
2. **OpenMP achieves the best CPU performance.** $\times 6.32$ speedup at 8 threads, 79% efficiency. Dynamic scheduling and thread reuse explain the advantage.
3. **Pthreads is competitive but less efficient.** $\times 5.67$ at 8T, 71% efficiency. Static row assignment and per-phase thread creation limit scalability.
4. **MPI achieves good speedup with communication overhead.** $\times 5.11$ at 8P, 64% efficiency. MPI would close the gap at multi-node scale.
5. **Hybrid performance inversely correlates with the inner thread count on a single node.** Hybrid 8×1 ($\times 4.92$, MPI+OpenMP) is the best hybrid configuration overall; Hybrid 1×8 is the worst ($\times 1.06$). The MPI+Pthreads variant follows the same shape, ranking 8×1 ($\times 4.84$) $>$ 4×2 ($\times 4.31$) $>$ 2×4 ($\times 1.99$) $>$ 1×8 ($\times 1.01$) — consistently a few percent slower than the OMP variant at each layout, due to the static partition and per-phase pthread create/join overhead. The hybrid advantage over pure OpenMP only emerges at multi-node scale.
6. **CUDA provides the largest speedup by far.** Benchmarked on the NVIDIA RTX 3050, the GPU completes similarity + prediction in **0.2205 s** — a **38.0×** speedup over the serial

baseline and $\approx 6\times$ faster than the best 8-thread CPU run. The similarity kernel launches 1,016,064 threads simultaneously.

Final ranking (single node):

CUDA GPU ($\times 38.0$) > OpenMP 8T ($\times 6.32$) > Pthreads 8T ($\times 5.67$) > MPI 8P ($\times 5.11$) > Hybrid 8 $\times$ 1 ($\times 4.92$) > Hybrid-PT 8 $\times$ 1 ($\times 4.84$) > Hybrid 4 $\times$ 2 ($\times 4.71$) > Hybrid-PT 4 $\times$ 2 ($\times 4.31$) > Hybrid 2 $\times$ 4 ($\times 2.11$) > Hybrid-PT 2 $\times$ 4 ($\times 1.99$) > Hybrid 1 $\times$ 8 ($\times 1.06$) > Hybrid-PT 1 $\times$ 8 ($\times 1.01$)

References

1. J. S. Breese, D. Heckerman, and C. Kadie, “Empirical analysis of predictive algorithms for collaborative filtering,” *arXiv:1301.7363*, 2013.
2. B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, “Item-based collaborative filtering recommendation algorithms,” in *Proc. WWW*, pp. 285–295, 2001.
3. B. Chapman, G. Jost, R. van der Pas, *Using OpenMP*. MIT Press, 2008.
4. W. Gropp, E. Lusk, A. Skjellum, *Using MPI*. MIT Press, 1994.
5. J. Nickolls et al., “Scalable parallel programming with CUDA,” *Queue*, vol. 6, no. 2, pp. 40–53, 2008.